Name: _____
Surname: _____
Date: _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. The following code asks the user for numbers and shows their factorials. However, it fails on line 5: "`TypeError: ">" not supported between instances of "str" and "int"`". Explain and fix the issue(s).

```
1  input = input("Give me a number: ")
2
3  while input != 0:
4      result = 1
5      while input > 0:
6          result = result * input
7          input -= 1
8
9      print(f"The factorial of your number is: result")
10     input = input("Give me another number: ")
```

Answer:

The first problem is that `input()` always returns a string, so we cannot compare it with an integer (line 5) or use it in a multiplication. We have to turn it into a number with `int()` when we read it. This also fixes line 3: a string is never equal to the integer 0, so the outer loop would never stop. *(+5 points)*

The second problem is the name of the variable. In line 1 we overwrite the built-in function `input` with the user's answer, so the second call to `input()` in line 10 fails (`'str' object is not callable`). We need a different variable name, for instance `number`. *(+5 points)*

The third problem is in the `print()` statement: the f-string contains the word `result` without curly brackets, so it prints the word "result" instead of its value. It must be {`result`}. *(+5 points)*

(Optional, fixed code)

```
1  number = int(input("Give me a number: "))
2
3  while number != 0:
4      result = 1
5      while number > 0:
6          result = result * number
7          number -= 1
8
9      print(f"The factorial of your number is: {result}")
10     number = int(input("Give me another number: "))
```

2. This code is intended to remove all occurrences of a specific value from a list. However, it doesn't remove all instances as expected. Identify and fix the issue(s).

```python
def remove_value(lst: list, value: int) -> list:
    for i in range(len(lst)):
        if lst[i] == value:
            lst.pop(i)
    return lst

numbers = [1, 2, 3, 2, 4, 2]
print(remove_value(numbers, 2))
# Expected Output: [1, 3, 4]
# Actual Output: [1, 3, 2, 4, 2]
```

Answer:

The problem is that we remove elements from the list while we are looping over its indices. Every time we do `lst.pop(i)`, all the following elements move one position to the left, so the element that comes right after the removed one is skipped and never checked. In addition, `range(len(lst))` was computed once at the beginning with the original length, so at the end the loop tries to access positions that no longer exist. *(+8 points)*

The easiest fix is not to modify the list we are iterating over: we create a new list and add only the elements that are different from `value`. *(+7 points)*

Comment: If we run the code as it is, we actually get an `IndexError: list index out of range` (the list ends up with 3 elements while the loop still goes up to index 5). Alternatively, the function can modify the list in place using a `while` loop with an index `i` that is only increased when nothing is removed.

(Optional, fixed code)

```python
def remove_value(lst: list, value: int) -> list:
    new_lst = []
    for item in lst:
        if item != value:
            new_lst.append(item)
    return new_lst

numbers = [1, 2, 3, 2, 4, 2]
print(remove_value(numbers, 2))  # [1, 3, 4]
```

3. The function below takes 1D-array and reshapes then to be squared. It works well for the first array, but the second one raises an error. Identify and fix the issue(s).

```python
import numpy as np

def square_shape(arr: np.array) -> np.array:
    if arr.ndim > 1:
        print("Only valid with 1D-arrays")
        return
    length = len(arr)
    size = length // 2
    return arr.reshape(size, size)

arr = np.array([1, 2, 3, 4])
print(square_shape(arr))
# Output: [[1, 2], [3, 4]]

arr = np.array([1, 2, 3, 4, 5, 6])
print(square_shape(arr))
# ValueError: cannot reshape array of size 6 into shape (3,3)
```

Answer:

The function assumes that the side of the square is half of the length of the array. This only works by coincidence with 4 elements (side 2). A square array of side s has $s \cdot s$ elements, so the side must be the **square root** of the length. With 6 elements, the function computes a side of 3, which would need 9 elements, and `reshape` fails. *(+5 points)*

We fix the calculation of the side using the square root: `size = int(np.sqrt(length))`. *(+5 points)*

Moreover, if the length is not a perfect square (like 6), no square shape is possible. We should check it, with `size * size != length`, and show a message and return nothing, just as the function already does for arrays that are not 1D. *(+5 points)*

(Optional, fixed code)

```python
import numpy as np

def square_shape(arr: np.array) -> np.array:
    if arr.ndim > 1:
        print("Only valid with 1D-arrays")
        return
    length = len(arr)
    size = int(np.sqrt(length))
    if size * size != length:
        print("The length is not a perfect square")
        return
    return arr.reshape(size, size)
```

4. The following code aims to take a subslice of the array "x". See what the user expects to get, and what is the actual output. Explain what is happening and fix the code so that we get the expected output.

```
import numpy as np

x = np.array(
  [[10, 20, 30],
   [40, 50, 60],
   [70, 80, 90]])
idx_rows = [0, 2]
idx_cols = [1, 2]
print(x[idx_rows, idx_cols])
# Expected output: [[20, 30], [80, 90]]
# Actual output:  [20, 90]
```

Answer:

When we index a NumPy array with two lists, `x[idx_rows, idx_cols]`, NumPy does not combine all rows with all columns. It pairs the lists element by element, so it picks the positions $(0, 1)$ and $(2, 2)$, which are the values 20 and 90. *(+8 points)*

To select the sub-matrix (every row of the first list with every column of the second one) we can use `np.ix_`, which builds that grid of indices: `x[np.ix_(idx_rows, idx_cols)]`. Another valid option is to index in two steps, `x[idx_rows][:, idx_cols]`. *(+7 points)*

Comment: In the statement, the last argument is written `idx_col`, a typo of `idx_cols`.

(Optional, fixed code)

```
import numpy as np

x = np.array(
    [[10, 20, 30],
     [40, 50, 60],
     [70, 80, 90]])
idx_rows = [0, 2]
idx_cols = [1, 2]
print(x[np.ix_(idx_rows, idx_cols)])
# [[20 30]
#  [80 90]]
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Removing Characters (20 points)

Create the `remove_first_char()` function to take a string and return a new version. This version should remove all occurrences of the first character, except the first one itself. For example, "sun shines" becomes "sun hine" (all "s" except the first were removed). Note that the function treats lowercase and uppercase as distinct characters.

Answer:

The function receives a string and returns a new string. Its first character is `text[0]`, and we store it in a variable, `first = text[0]`. *(+5 points)*

All the characters after the first one are `text[1:]`. On that part of the string only, we remove every occurrence of the first character using the string method `replace`: `text[1:].replace(first, "")`. This method is case sensitive, so uppercase and lowercase letters are treated as different characters, as required. *(+8 points)*

Finally we concatenate the first character and the cleaned rest, and **return** the result (strings are immutable, so `replace` gives a new string and never modifies the original). *(+4 points)*

We should also take care of the empty string: `text[0]` would raise an error, so we first check `if len(text) == 0` and return the empty string. *(+3 points)*

(Optional, code)

```python
def remove_first_char(text: str) -> str:
    # Empty string: nothing to do
    if len(text) == 0:
        return text
    first = text[0]
    # Remove the first character from the rest of the string
    rest = text[1:].replace(first, "")
    return first + rest

print(remove_first_char("sun shines"))  # sun hine
```

6. Average Value in Merged Dictionary (20 points)

Create a Python function called `dict_merge()` that merges a list of dictionaries into one unified dictionary. This function should assume that all input dictionaries exclusively contain numeric values. *(10 points)*
In cases where the same key is present in multiple dictionaries, compute the final value for this key in the resultant merged dictionary as the average of its values. *(5 points)*
Ensure that the final output values are integers if they do not have a fractional component. *(5 points)*

```
# For instance
dict_1 = {"a": 1, "b": 2}
dict_2 = {"a": 2, "c": 3}
dict_out = dict_merge(dict_1, dict_2)
print(dict_out)
# Output: {a: 1.5, b: 2, c: 3}
```

Answer:

We define a function that accepts any number of dictionaries, `def dict_merge(*dicts)`, and loops over them. To be able to compute averages later, we initialize two empty dictionaries: `totals`, for the sum of the values of each key, and `counts`, for the number of dictionaries where each key appears. Inside a nested loop `for key, value in dictionary.items()`, if the key is not yet in `totals`, we initialize both dictionaries to 0 for that key. Then we add the value to `totals[key]` and increase `counts[key]` by 1. This way, all the keys of all the dictionaries end up in the merged result. *(+10 points)*

Then, we create the output dictionary looping over the keys of `totals`, and we store as value the average, `totals[key] / counts[key]`. For a key that appears in only one dictionary, the average is just its own value. *(+5 points)*

An average is always a float (`2 / 1` is `2.0`), so before storing it we check whether it has a fractional part: if `average == int(average)`, we convert it with `int()`. *(+5 points)*

Comment: The example calls the function with two separate dictionaries, which is why we use `*dicts`. If we have the dictionaries in a list, we can call it with `dict_merge(*list_of_dicts)`.

(Optional, code)

```
def dict_merge(*dicts):
    # Sum of the values and number of appearances of each key
    totals = {}
    counts = {}
    for dictionary in dicts:
        for key, value in dictionary.items():
            if key not in totals:
                totals[key] = 0
                counts[key] = 0
            totals[key] += value
            counts[key] += 1
    # Average of each key
    merged = {}
    for key in totals:
        average = totals[key] / counts[key]
        # Integer if there is no fractional component
        if average == int(average):
            average = int(average)
        merged[key] = average
    return merged
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.